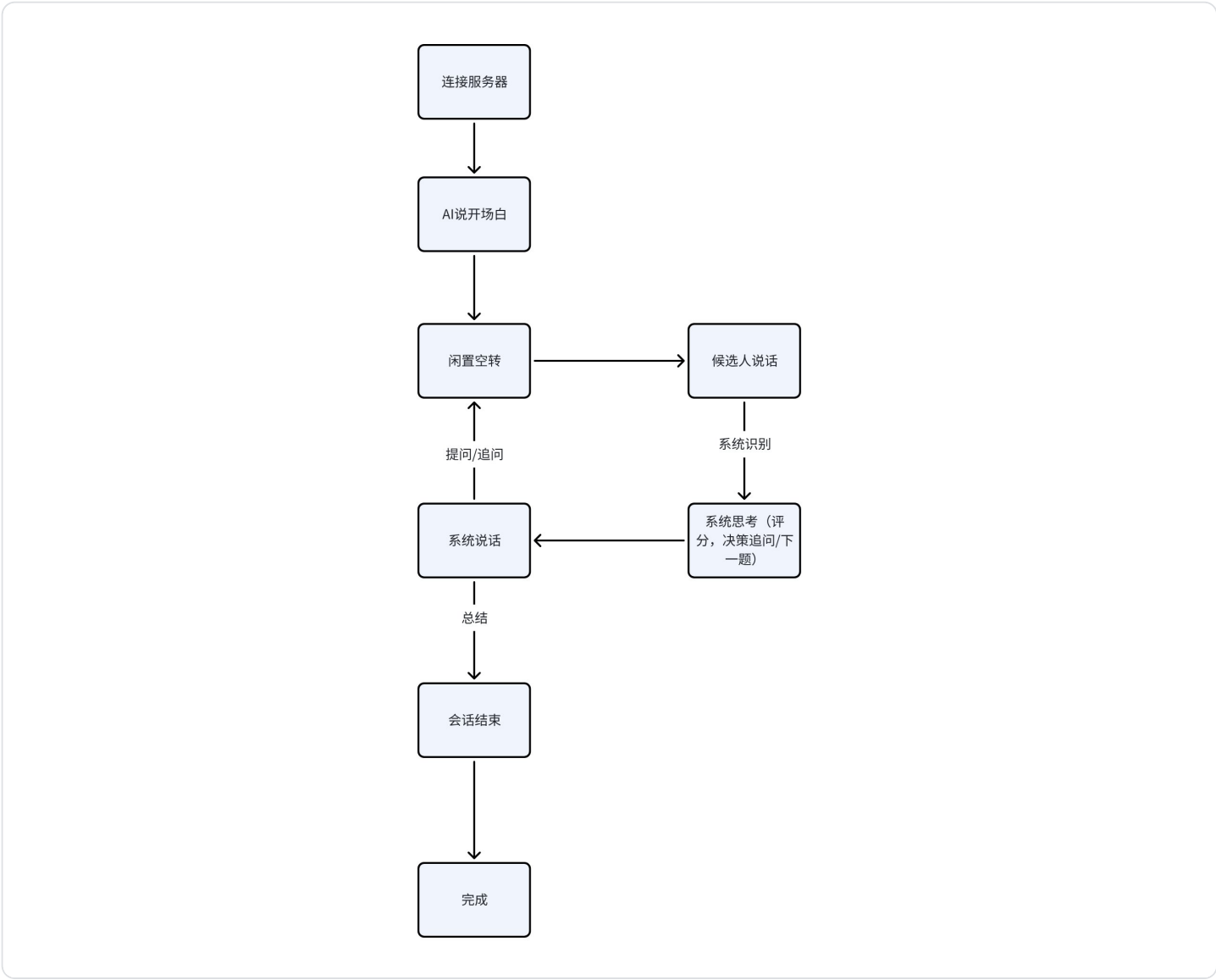
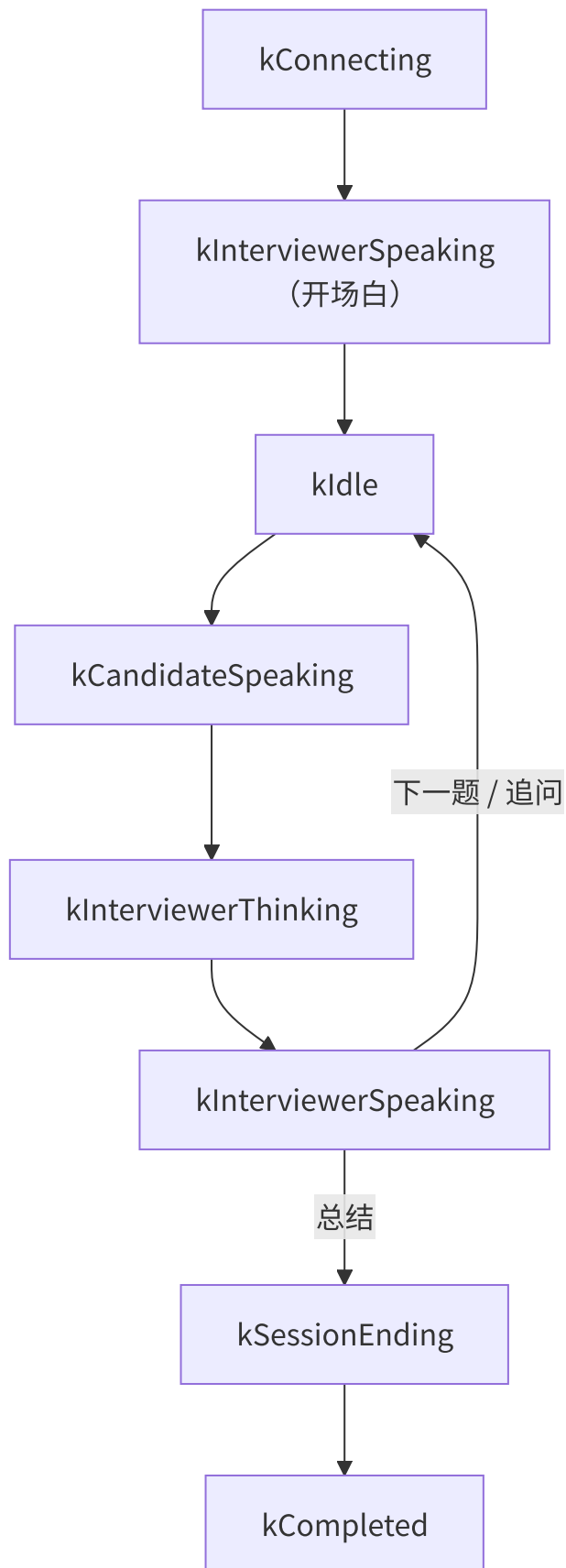


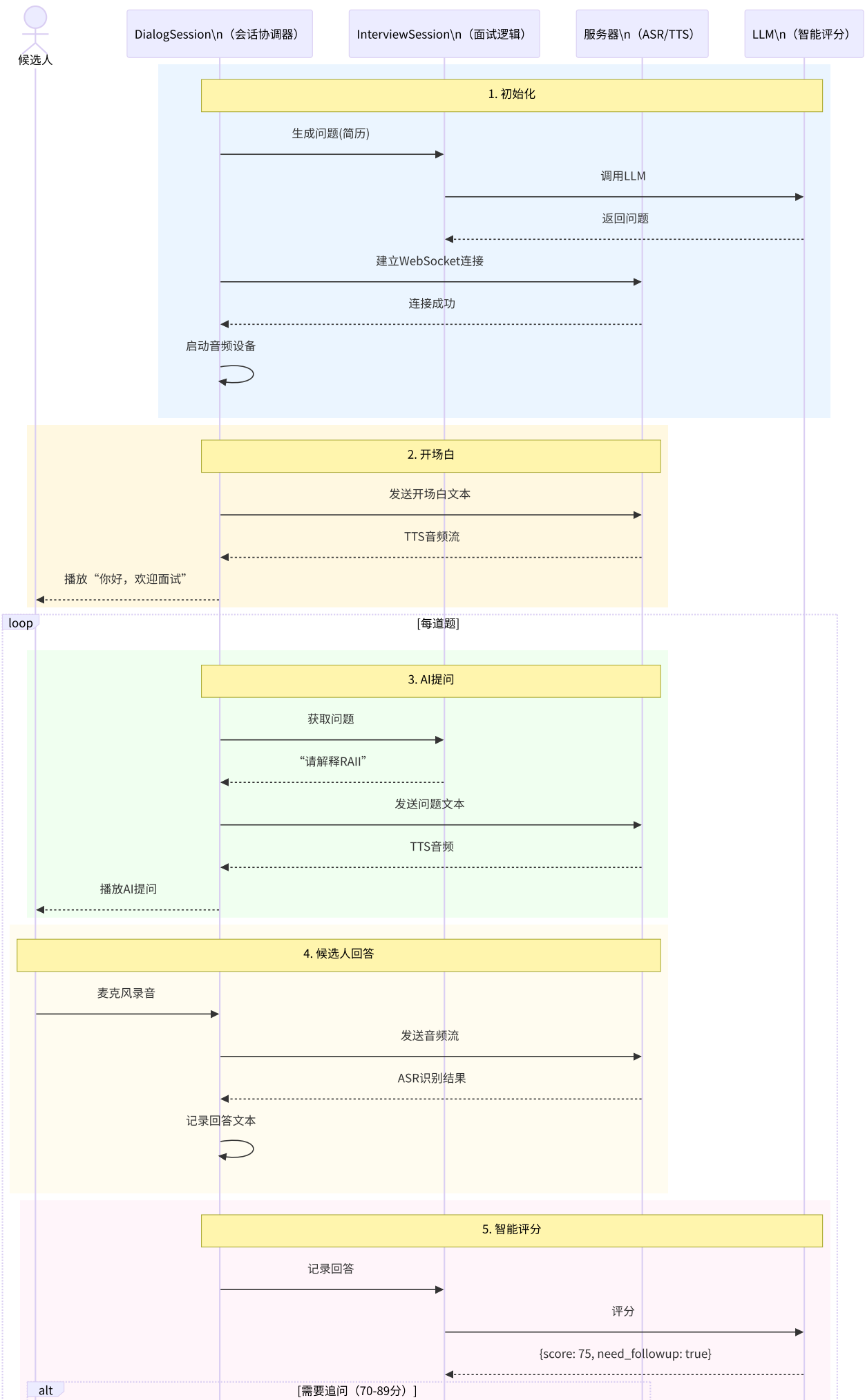
AI面试

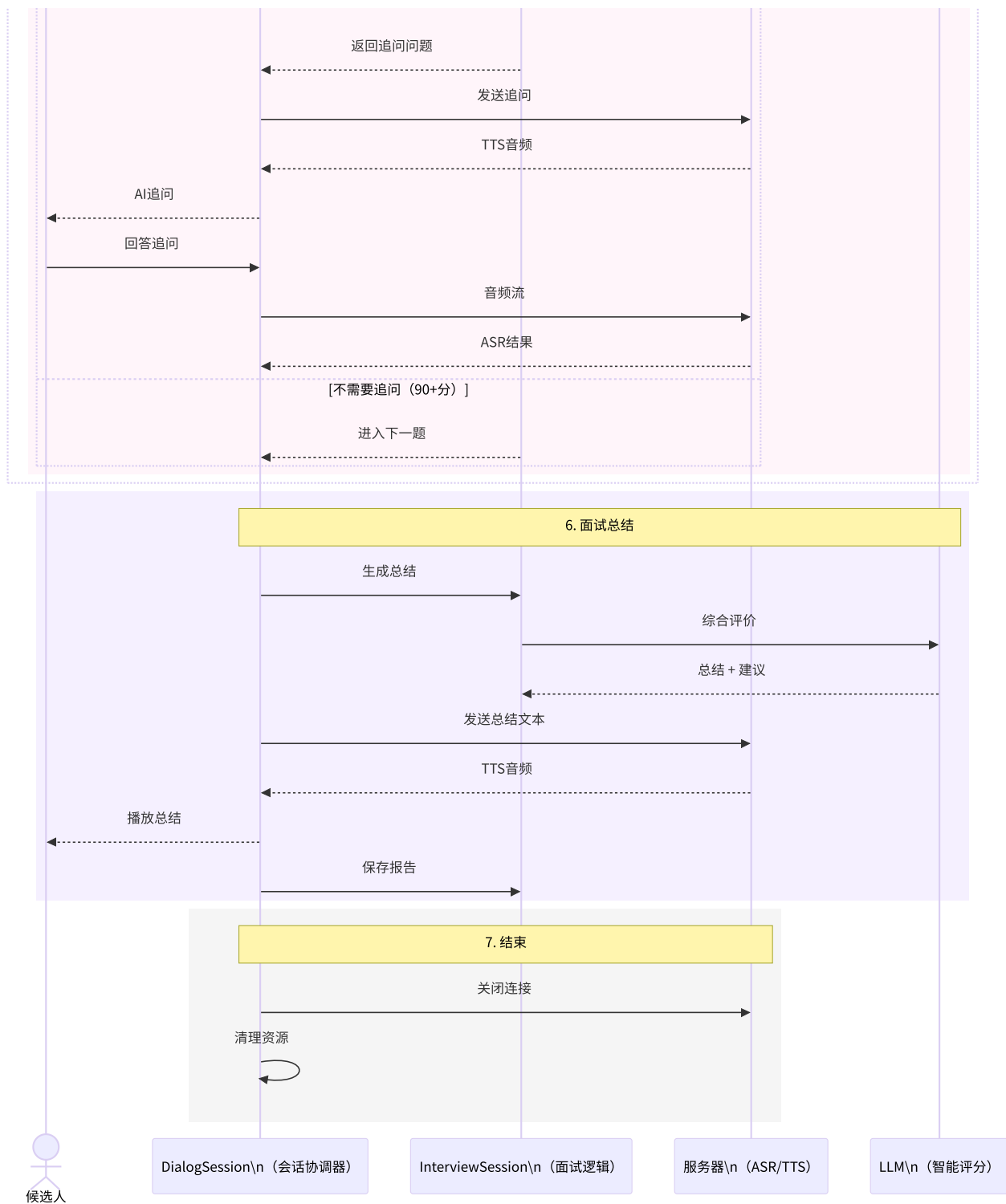
项目架构



状态机图







应用层

- UI：负责展示和用户交互

会话层

- DialogSession：负责对话会话协调、上下文管理
- InterviewManager：负责面试流程管理、业务推进

服务层

- RealtimeClient: WebSocket 实时通信
- AudioManager: 音频输入输出
- LLMClient: 大模型调用、AI评分
- PDFParser: 简历解析

基础层

- Config: 配置管理
- Protocol: 协议编解码
- Logger: 日志系统
- Utils: 工具函数

会话协调器 (dialog session)

生命周期管理: 启动、运行、停止

事件分发: 接收服务器事件, 触发业务逻辑

状态转换: 维护面试状态机

资源协调: 管理音频、网络、面试流程

面试逻辑 InterviewSession

问题管理: 生成、存储、按序提供

回答记录: 保存回答和评分

智能评分: 调用 LLM 评估质量

追问决策: 判断是否深入提问

报告生成: 生成结构化 JSON 报告

ASR

Speech -> Text

语音转文字

TTS

Text -> Speech

文字转语音

生成问题-> 面试逻辑-> LLM->返回n题

建立websocket连接, 启用音频

开场白-> 音频-> 播放给面试者

获取问题-》面试逻辑-》发送给TTS转成音频->AI播放给面试者

候选人回答-》dialog-》asr-》记录文本-》面试逻辑-》LLM评分-》追问或者下一题

项目流程设计

Code block

```
1  开始面试
2      Dialogsession -> 建立WebSocket连接, 启动音频设备
3      InterviewSession 结合简历/配置生成首题
4      Dialogsession 发送开场白, TTS 播放
5      进入题目循环
6
7  题目循环
8      InterviewSession 提供当前题目
9      DialogSession 发送题目文本给 ARS/TTS 服务
10     TTS 播放 AI 提问
11     候选人回答
12     DialogSession 采集语音并发送音频流
13     ASR 返回识别文本
14     DialogSession 记录回答文本并提交给InterviewSession
15     InterviewSession 调用LLM并进行评分
16     LLM 返回评分结果和是否需要追问
17     如果需要追问
18         InterviewSession 生成追问
19         DialogSession 播放追问
20         候选人再回答
21         ASR再识别
22         InterviewSession 再记录
23     如果不需要追问
24         进入下一题
25 所有题目完成
26     InterviewSession 生成面试总结与结构化报告
27     DialogSession 发送总结文本, TTS 播放总结
28     保存报告
29     关闭连接
30     清理资源
31     面试完成
32
```

Code block

```
1  1. [PDFParser] 提取简历文本
2  2. [LLMClient] 生成面试问题
```

```
3  3. [RealtimeClient] 连接服务器，发送开场白
4  4. [AudioManager] 启动麦克风和播放线程
5  5. [RealtimeClient] 发送问题-> AI说话
6  6. [AudioManager] 播放TTS 音频，关闭麦克风
7  7. [AudioManager] 打开麦克风，录制候选人回答
8  8. [RealtimeClient] 发送音频流 -> ASR 识别
9  9. [LLMClient] 评估回答，决定追问/下一题
10 重复步骤 5-9，直到所有问题完成
11 [LLMClient] 生成面试总结
12 [RealtimeClient] 关闭连接
```

Code block

```
1  开始面试
2
3      DialogSession -> 通过 RealtimeClient 建立 WebSocket 连接
4      AudioManager 启动音频设备，初始化麦克风与播放线程
5      PDFParser 提取简历文本
6      InterviewSession 结合简历 / 配置准备面试上下文
7      LLMClient 生成首题 / 初始题目列表
8      DialogSession 发送开场白文本
9      RealtimeClient 将开场白发送到 ASR/TTS 服务
10     TTS 返回开场白音频
11     AudioManager 播放开场白，必要时关闭麦克风避免回采
12     进入题目循环
13
14
15  题目循环
16
17     InterviewSession 提供当前题目
18     DialogSession 组织题目消息
19     RealtimeClient 发送题目文本到 ASR/TTS 服务
20     TTS 返回题目音频
21     AudioManager 播放 AI 提问，播放期间关闭麦克风
22     候选人开始回答
23     AudioManager 打开麦克风，录制候选人语音
24     DialogSession 采集语音数据
25     RealtimeClient 发送音频流到 ASR 服务
26     ASR 返回识别文本
27     DialogSession 记录回答文本，并提交给 InterviewSession
28     InterviewSession 保存回答记录
29     LLMClient 对回答进行评分
30     LLM 返回评分结果，并给出是否需要追问 / 是否进入下一题的决策
31
32     如果需要追问，重复题目循环步骤
33     InterviewSession 生成追问问题
```



```
34         DialogSession 组织追问消息
35         RealtimeClient 发送追问文本到 ASR/TTS 服务
36         TTS 返回追问音频
37         AudioManager 播放追问，必要时关闭麦克风
38         候选人再次回答
39         AudioManager 打开麦克风并录音
40         DialogSession 采集追问回答语音
41         RealtimeClient 再次发送音频流到 ASR 服务
42         ASR 返回追问识别文本
43         DialogSession 记录追问回答文本，并提交给 InterviewSession
44         InterviewSession 再次保存 / 更新回答记录
45         再根据结果决定是否继续追问或进入下一题
46
47     如果不需要追问
48         进入下一题
49
50     重复上述流程，直到所有题目完成
51
52
53 所有题目完成
54
55     InterviewSession 汇总整场面试记录
56     LLMClient 生成面试总结、综合评价与改进建议
57     InterviewSession 生成结构化报告
58     DialogSession 组织总结消息
59     RealtimeClient 发送总结文本到 ASR/TTS 服务
60     TTS 返回总结音频
61     AudioManager 播放总结
62     保存报告
63     RealtimeClient 关闭连接
64     AudioManager 停止录音与播放线程
65     DialogSession 清理资源
66     面试完成
```

环境搭建

豆包端到端实时语音大模型

<https://console.volcengine.com/speech/service/10017?AppID=3577842255&projectName=default>

- API ID: 3577842255
- Access Token: Gj9CHGomsZxq8yYplgfrKBYmHTlx3PVb

小马算力

- API URL(默认) :<https://api.tokenpony.cn/v1/chat/completions>
- API Key: sk-aac063355dc94a0894ac1c375a9bb19c

基础安装

Code block

```
1  git clone xxxxxxxxx
2  git checkout -b feature/rpc_init // 创建新分支, feature/rpc_init是分支名
3  git branch // 查看分支
4
5  git add -u // 添加所有修改的文件
6
7  git commit -m "提交的描述" // feat新功能 fix bug 修改bug
8  git commit -amend // 修改上一个commit
9
10 git push origin feature/rpc_init // 这个远程库名字叫origin
```

Vcpkg安装

创建vcpkg.json

Code block

```
1  {
2    "name": "ai-interview",
3    "version": "1.0.0",
4    "description": "C++ Technical Interview System with Real-time Audio Dialog (Qt GUI)",
5    "dependencies": [
6      "openssl",
7      "boost-system",
8      "boost-thread",
9      "boost-asio",
10     "boost-beast",
11     "nlohmann-json",
12     "zlib",
13     "portaudio",
14     "curl",
15     "podofo",
16     "spdlog",
```

```

17     "qtbase"
18 ],
19     "overrides": [
20         {
21             "name": "openssl",
22             "version": "3.6.0"
23         },
24         {
25             "name": "nlohmann-json",
26             "version": "3.12.0"
27         },
28         {
29             "name": "zlib",
30             "version": "1.3.1"
31         },
32         {
33             "name": "portaudio",
34             "version": "19.7"
35         },
36         {
37             "name": "curl",
38             "version": "8.16.0"
39         },
40         {
41             "name": "podofo",
42             "version": "1.0.2#1"
43         },
44         {
45             "name": "spdlog",
46             "version": "1.16.0"
47         },
48         {
49             "name": "qtbase",
50             "version": "6.9.1"
51         }
52     ],
53     "builtin-baseline": "63bb8e44c140791580201d29c0c16985a88969cf"
54 }
55

```

- 网络通信: OpenSSL Boost.Asio/Beast
- 音频处理: PortAudi
- 界面框架: qtbase
- 数据处理: nlohmann-json、zlib

- 日志系统: spdlog
- HTTP客户端: libcurl
- PDF解析: podofo

Code block

```
1 // 添加baseline
2 $VCPKG_ROOT/vcpkg x-update-baseline --add-initial-baseline
```

安装依赖，

过程会出现很多问题。如果出现连接git不稳定问题，可以下载对应链接到本地再拷贝到远程库中。缺少相关工具库，就下载工具库之后再继续安装。

Code block

```
1 $VCPKG_ROOT/vcpkg install
```

创建最小 CMakeLists.txt

接下来要把 `vcpkg` 和 `CMake` 接起来。

在项目根目录创建：

Code block

```
1 touch CMakeLists.txt
```

写入下面内容：

Code block

```
1 cmake_minimum_required(VERSION 3.16)
2
3 project(ai_interview LANGUAGES CXX)
4
5 set(CMAKE_CXX_STANDARD 17)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7 set(CMAKE_CXX_EXTENSIONS OFF)
8
9 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
10
11 if(NOT CMAKE_BUILD_TYPE)
12     set(CMAKE_BUILD_TYPE Debug)
```

```
13     endif()
14
15     find_package(nlohmann_json CONFIG REQUIRED)
16     find_package(spdlog CONFIG REQUIRED)
17
18     add_executable(ai_interview
19         src/main.cc
20     )
21
22     target_link_libraries(ai_interview PRIVATE
23         nlohmann_json::nlohmann_json
24         spdlog::spdlog
25     )
```

创建最小 main.cc

在 `src` 目录下创建：

Code block

```
1 touch src/main.cc
```

写入以下内容：

Code block

```
1 #include <iostream>
2
3 #include "nlohmann/json.hpp"
4 #include "spdlog/spdlog.h"
5
6 int main() {
7     nlohmann::json message;
8     message["project"] = "ai_interview";
9     message["status"] = "ok";
10
11     spdlog::info("ai_interview start");
12     std::cout << message.dump(4) << '\n';
13
14     return 0;
15 }
```

用 CMake 配置项目

现在开始真正测试环境。

执行：

Code block

```
1  cmake -B build -S . -  
    DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake
```

这一步的意思是：

- `-B build`：把构建文件放到 `build/`
- `-S .`：当前目录就是源码根目录
- `-DCMAKE_TOOLCHAIN_FILE=...`：告诉 CMake 去用 vcpkg 的工具链

如果这一步成功，说明：

- `CMakeLists.txt` 没问题
- `vcpkg` 已经接上了
- `find_package` 能找到库了

编译项目

执行：

Code block

```
1  cmake --build build -j
```

如果成功，你会在 `build` 目录里得到可执行文件。

运行程序

执行：

Code block

```
1  ./build/ai_interview
```

如果输出类似：

Code block

```
1  [info] ai_interview start  
2  {
```

```
3     "project": "ai_interview",
4     "status": "ok"
5 }
```

使用git add前一定要记得先添加.gitignore文件

Code block

```
1  cat > .gitignore <<'EOF'
2  # Build directories
3  build/
4  cmake-build-*/
5  out/
6
7  # CMake generated files
8  CMakeCache.txt
9  CMakeFiles/
10 cmake_install.cmake
11 compile_commands.json
12 CTestTestfile.cmake
13 Testing/
14
15 # vcpkg installed dependencies
16 vcpkg_installed/
17
18 # Makefile generated by CMake
19 Makefile
20
21 # Executables and binaries
22 *.out
23 *.exe
24 *.o
25 *.obj
26 *.so
27 *.a
28 *.lib
29 *.dll
30
31 # Logs
32 *.log
33
34 # Editor / IDE
35 .vscode/
36 .idea/
37
38 # OS files
```

```
39  .DS_Store
40  Thumbs.db
41
42  # Environment / secrets
43  .env
44  .env.*
45  config.local.json
46  secrets.json
47  EOF
```

配置文件修改

就是在config下创建配置文件

Code block

```
1  vim config/default_config.json
```

1. 飞书豆包语音服务配置(WebSocket部分)

Code block

```
1  "ws": {
2      "base_url": "wss://openspeech.bytedance.com/api/v3/realtime/dialogue",
3      "headers": {
4          "X-API-App-ID": "你的飞书App ID",
5          "X-API-Access-Key": "你的飞书Access Token",
6          "X-API-Resource-Id": "volc.speech.dialog",
7          "X-API-App-Key": "你的飞书App Key",
8          "X-API-Connect-Id": ""
9      }
10 }
```

2. 大语言模型（小马算力）

Code block

```
1  "llm": {
2      "api_url": "https://api.tokenpony.cn/v1/chat/completions",
3      "api_key": "sk-aac063355dc94a0894ac1c375a9bb19c",
4      "model": "qwen3-8b",
5      "temperature": 0.3,
6      "max_tokens": 32000,
```



```
7     "timeout_seconds": 60
8 },
```

3. TTS/ARS

Code block

```
1  "tts": {
2      "speaker": "zh_male_yunzhou_jupiter_big tts",
3      "channel": 1,
4      "format": "pcm",
5      "sample_rate": 24000
6  },
7  "asr": {
8      "end_smooth_window_ms": 2000,
9      "vad_silence_duration": 2500,
10     "vad_speech_trigger_duration": 400
11 },
```

根据build.py编译运行

Code block

```
1  // ubuntu下用的python3
2  // 基本编译
3  python3 build.py
4
5  // 或者调试模式编译
6  python3 build.py --config Debug
```

CMake使用

Code block

```
1  ~/.bashrc
2  export VCPKG_ROOT=/path/to/vcpkg
3
4  cmake -S . -B build \
5      -DCMAKE_TOOLCHAIN_FILE=$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake
```

Code block

```
1 // 配置
2 cmake -S . -B build
3 // 编译
4 cmake --build build -j
```

Github